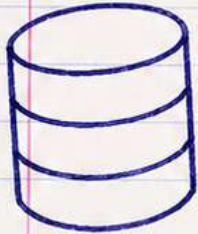
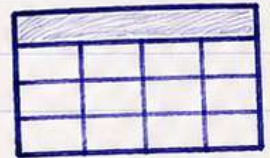


AI Simplified ❤️



30 SQL



INTERVIEW QUESTIONS

— EVERY DATA ANALYST —
= MUST KNOW =

- | | |
|---|--|
| ☒ SQL Interview Questions | ☒ Basics to Advanced |
| ☒ Detailed Answers | ☒ Perfect for Interviews |
| ☒ Real-world Scenarios | ☒ For Data Analysts |
| ☒ Cheat Sheet | ☒ For Developers |
| ☒ JOINS Cheat Sheet | ☒ For All SQL Learners |



Learn SQL.
Crack Interviews.
Build Your Career.



— AI SIMPLIFIED —
BY M MANI CHANDANA



SQL INTERVIEW QUESTIONS

AI Simplified ♥

- ① Q: Write a SQL query to retrieve all columns from the 'Employees' table.

Answer: `SELECT *`
`FROM Employees;` → The * retrieves all columns from the table.
- ② Q: Write a SQL query to find all employees who earn more than 50,000.

Answer: `SELECT *`
`FROM Employees`
`WHERE Salary > 50000;` → The WHERE clause filters records based on conditions.
- ③ Q: Write a SQL query to get the names and departments of all employees.

Answer: `SELECT Name, Department`
`FROM Employees;` → We can specify the required columns instead of using *.
- ④ Q: Write a SQL query to find the total number of employees in the company.

Answer: `SELECT COUNT(*)`
`FROM Employees;` → COUNT(*) returns the total number of rows in the table.
- ⑤ Q: Write a SQL query to find the average salary of employees.

Answer: `SELECT AVG(Salary)`
`FROM Employees;` → AVG() is an aggregate function used to calculate average.

SQL INTERVIEW QUESTIONS

AI Simplified ♥

- ⑥ Q: Write a SQL query to find the second highest salary from the Employees table.

Answer:

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (SELECT MAX(Salary)
                FROM Employees);
```

→ The subquery finds the highest salary first, then we get the next maximum.

- ⑦ Q: Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

```
SELECT e.Name, e.Department, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
                 FROM Employees
                 WHERE Department = e.Department);
```

→ Correlated subquery compares salary with department-wise average.

- ⑧ Q: Write a SQL query to find the number of employees who joined each year.

Answer:

```
SELECT YEAR(JoinDate) AS JoinYear, COUNT(*) AS EmpCount
FROM Employees
GROUP BY YEAR(JoinDate)
ORDER BY JoinYear;
```

→ GROUP BY with YEAR() groups employees by joining year.

- ⑨ Q: Write a SQL query to find the top 3 departments based on average salary.

Answer:

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
ORDER BY AvgSalary DESC
LIMIT 3;
```

→ Calculates average salary for each department and returns top 3.

- ⑩ Q: Write a SQL query to find employees who have not been assigned to any project.

Answer:

```
SELECT e.Name
FROM Employees e
LEFT JOIN Projects p ON e.EmpID = p.EmpID
WHERE p.EmpID IS NULL;
```

→ LEFT JOIN + IS NULL helps to find non-matching records.

SQL INTERVIEW QUESTIONS

AI Simplified ♥

- 11 **Q:** Write a SQL query to find employees who have a salary greater than the average salary of the company.
- Answer:**
- ```
SELECT Name, Salary
FROM Employees
WHERE Salary > (SELECT AVG(Salary)
 FROM Employees);
```
- Subquery finds the average salary first, then outer query compares each employee's salary with it.
- 12 **Q:** Write a SQL query to find employees who earn more than their department average salary.
- Answer:**
- ```
SELECT e.Name, e.Department, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
                FROM Employees
                WHERE Department = e.Department);
```
- Compares each employee's salary with the average salary of their respective department.
- 13 **Q:** Write a SQL query to find duplicate email IDs in the Employees table.
- Answer:**
- ```
SELECT Email, COUNT(*) AS Cnt
FROM Employees
GROUP BY Email
HAVING COUNT(*) > 1;
```
- Groups by Email and filters those having count greater than 1.
- 14 **Q:** Write a SQL query to find employees who have worked in more than one department.
- Answer:**
- ```
SELECT e.Name
FROM Employees e
GROUP BY e.Name
HAVING COUNT(DISTINCT e.Department) > 1;
```
- DISTINCT counts unique departments per employee.
- 15 **Q:** Write a SQL query to find the nth highest salary from the Employees table.
- Answer:**
- ```
SELECT MAX(Salary) AS Nth_Highest_Salary
FROM Employees
WHERE Salary < (SELECT MAX(Salary)
 FROM Employees
 LIMIT n-1, 1); -- n = position
```
- Inner query gets the (n-1)<sup>th</sup> highest salary, outer query gives the next highest (nth).

# SQL INTERVIEW QUESTIONS

AI Simplified ♥

16 Q: Write a SQL query to find the customers who have placed orders but have never made a payment.

Answer: SELECT DISTINCT c.CustomerID, c.CustomerName  
FROM Customers c  
JOIN Orders o ON c.CustomerID = o.CustomerID  
LEFT JOIN Payments p ON o.OrderID = p.OrderID  
WHERE p.PaymentID IS NULL;

→ LEFT JOIN helps to find orders that do not have matching payments (i.e., unpaid orders).

17 Q: Write a SQL query to find the moving average of sales amount for each day (considering current day and previous 2 days).

Answer: SELECT OrderDate, SUM(SalesAmount) AS TotalSales,  
AVG(SUM(SalesAmount)) OVER (ORDER BY OrderDate  
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS MovingAvg  
FROM Sales  
GROUP BY OrderDate  
ORDER BY OrderDate;

→ Window function to calculate 3-day moving average.

18 Q: Write a SQL query to find the 2nd highest salary from the Employees table (without using LIMIT OFFSET).

Answer: SELECT MAX(Salary) AS Second\_Highest\_Salary  
FROM Employees  
WHERE Salary < (SELECT MAX(Salary) FROM Employees);

→ First, get the highest salary. Then find the maximum below that.

19 Q: Write a SQL query to find employees whose salary is higher than the average salary of their department.

Answer: SELECT e.Name, e.Department, e.Salary  
FROM Employees e  
WHERE e.Salary > (SELECT AVG(Salary)  
FROM Employees  
WHERE Department = e.Department);

→ Correlated subquery compares each employee's salary with their department average.

20 Q: Write a SQL query to find the top 3 customers who have spent the most amount in total.

Answer: SELECT CustomerID, SUM(Amount) AS TotalSpent  
FROM Orders  
GROUP BY CustomerID  
ORDER BY TotalSpent DESC  
LIMIT 3;

→ Groups by customer and sums total spent, then picks top 3.

# SQL INTERVIEW QUESTIONS

AI Simplified ♥

21 Q: Write a SQL query to find the departments that have no employees.

Answer:

```
SELECT d.Department
FROM Departments d
LEFT JOIN Employees e ON d.DepartmentID = e.DepartmentID
WHERE e.EmployeeID IS NULL;
```

→ LEFT JOIN returns all departments and NULLs for employees. We filter those NULLs.

22 Q: Write a SQL query to find employees who earn more than the average salary of their department.

Answer:

```
SELECT e.Name, e.Salary, e.DepartmentID
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
 FROM Employees
 WHERE DepartmentID = e.DepartmentID);
```

→ Correlated subquery gets average salary of the same department and compares with employee's salary.

23 Q: Write a SQL query to find the employees who have the same salary in the same department.

Answer:

```
SELECT e1.Name, e1.DepartmentID, e1.Salary
FROM Employees e1
JOIN Employees e2
ON e1.DepartmentID = e2.DepartmentID
AND e1.Salary = e2.Salary
AND e1.EmployeeID <> e2.EmployeeID;
```

→ Self join on same department and salary, excluding same employee using <>.

24 Q: Write a SQL query to find the nth highest salary from the Employees table (n = 3).

Answer:

```
SELECT MAX(Salary) AS Third_Highest_Salary
FROM Employees
WHERE Salary < (SELECT DISTINCT Salary
 FROM Employees
 ORDER BY Salary DESC
 LIMIT 2,1); -- 2 rows offset, 1 row
```

→ Inner query gets the 3rd highest distinct salary. Outer query returns it.

25 Q: Write a SQL query to get the cumulative (running) total of salary in each department based on hire date.

Answer:

```
SELECT Name, DepartmentID, HireDate, Salary,
SUM(Salary) OVER (PARTITION BY DepartmentID
 ORDER BY HireDate
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM Employees;
```

→ Window function calculates running total of salary within each department ordered by hire date.

# SQL INTERVIEW QUESTIONS

AI Simplified ♥

26 Q: Write a SQL query to find the month-over-month growth of total sales.

Answer: 

```
SELECT MONTH(OrderDate) AS Month,
SUM(Sales) AS TotalSales,
LAG(SUM(Sales)) OVER (ORDER BY MONTH(OrderDate)) AS PrevMonthSales,
ROUND((SUM(Sales) - LAG(SUM(Sales)) OVER (ORDER BY MONTH(OrderDate)))
/ LAG(SUM(Sales)) OVER (ORDER BY MONTH(OrderDate)) * 100, 2) AS MoM_Growth_pct
FROM Sales
GROUP BY MONTH(OrderDate)
ORDER BY Month;
```

→ LAG() gets previous month's sales to calculate month-over-month growth %.

27 Q: Write a SQL query to find the top 5 products by revenue in each category.

Answer: 

```
SELECT * FROM (
SELECT Category, ProductID, SUM(Price * Quantity) AS Revenue,
ROW_NUMBER() OVER (PARTITION BY Category ORDER BY SUM(Price * Quantity) DESC) AS rn
FROM Sales
GROUP BY Category, ProductID
) t
WHERE rn <= 5
ORDER BY Category, Revenue DESC;
```

→ ROW\_NUMBER() ranks products within each category by revenue.

28 Q: Write a SQL query to find the customers who have placed orders in consecutive 3 months.

Answer: 

```
SELECT CustomerID
FROM (
SELECT CustomerID, MONTH(OrderDate) AS Month,
ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY MONTH(OrderDate)) AS rn
FROM Orders
GROUP BY CustomerID, MONTH(OrderDate)
) t
GROUP BY CustomerID
HAVING COUNT(DISTINCT Month) = 3 AND MAX(Month) - MIN(Month) = 2;
```

→ Checks if a customer has orders in 3 consecutive months (difference = 2).

29 Q: Write a SQL query to find the first order date and the last order date for each customer.

Answer: 

```
SELECT CustomerID,
MIN(OrderDate) AS FirstOrderDate,
MAX(OrderDate) AS LastOrderDate
FROM Orders
GROUP BY CustomerID;
```

→ MIN() gives first order date and MAX() gives last order date.

30 Q: Write a SQL query to find employees who earn more than the average salary of employees in their department.

Answer: 

```
SELECT e.Name, e.Department, e.Salary
FROM Employees e
WHERE e.Salary > (SELECT AVG(Salary)
FROM Employees
WHERE Department = e.Department);
```

→ Correlated subquery compares employee salary with department average.